TRAINING · RAN OPTIMISATION

AI-Powered RAN Trial Analysis

with Claude Cowork & Claude Code

Automated KPI statistics, causal-chain analysis, and technical memo generation for 4G LTE and 5G NR NSA/EN-DC Nokia RAN optimization trials

Andriy Korobeyko · 2026

Agenda

01 **Claude Ecosystem**
Claude, Claude Code, Cowork — what they are

02 **Installation**
Windows setup guide step-by-step

03 **Cowork vs Claude Code**
When to use which tool

04 **Skills & Project Structure**
Concepts, folder layout, logical flow

05 **RAN Trial Workflow**
LTE and NR/EN-DC pipeline overview

06 **SKILL & Methodology Files**
File-by-file walkthrough

07 **How to Use It**
Running the pipeline, expected outputs

08 **Tips & Next Steps**
Best practices, extending to new trials

PART 1

The Claude Ecosystem

Claude · Claude Code · Claude Cowork

What is Claude?

An AI assistant built by Anthropic

Claude is a large language model (LLM) created by Anthropic.

Unlike a search engine, Claude reasons, writes, codes, and analyses — it understands context across a full conversation and can take multi-step actions on your behalf.

This workflow uses **Claude Sonnet 4.6** — with balance of reasoning depth and speed for RAN KPI analysis and script generation.

Claude Haiku

Fast, lightweight — great for simple tasks

Claude Sonnet

Balanced speed and intelligence — daily workhorse

Claude Opus

Most capable — complex reasoning and analysis

Access methods: Web app · Mobile · API · Claude Code (CLI) · Claude Cowork (Desktop)

What is Claude Code?

An AI assistant built by Anthropic

Claude Code is a terminal-based agent that runs directly in your shell. It can read, write, and edit files, run commands, install packages, and complete multi-step engineering tasks autonomously.

```
# Install
npm install -g @anthropic-ai/claude-code

# Launch in any project folder
claude

# Or with a one-shot task
claude "analyse these KPI files and plot sigma"
```

Designed for	Developers, engineers running code-heavy workflows
Works in	Any terminal — Windows (WSL/cmd), macOS, Linux
Key power	Full shell access, runs scripts, installs libs, reads files
Plugins	Installable MCP bundles extend capabilities
This workflow	Claude Code runs the Python/Node scripts for you

claude.ai/code | Requires Node.js 18+

What is Claude Cowork?

A desktop AI workspace for non-developers

Claude Cowork is a desktop application (Windows/macOS) that gives professionals access to Claude's full agentic capabilities — file operations, web research, document creation, and custom workflows — without using a terminal.

File Access

- Read/write files in a connected folder
- Edit Excel, Word, PDF, code files
- Run Python & Node scripts via sandbox

Skills System

- Install pre-built Skills (like this RAN workflow)
- Skills contain SKILL.md instructions + scripts + references
- Claude reads them before acting

MCP Connectors

- Connect to Slack, Asana, Gmail, databases
- Fetch live data into analysis
- Extensible plugin marketplace

Persistent Memory

- Remembers your preferences across sessions
- Stores project context
- No need to re-explain each time

Claude Products — at a Glance

	Claude.ai (Web/Mobile)	Claude Code (CLI)	Claude Cowork (Desktop)
Target user	Anyone	Developers / engineers	Engineers, analysts
Interface	Browser / phone	Terminal	Desktop GUI
File access	Upload only	Full filesystem (shell)	Connected folder
Run scripts	No	Yes — direct shell	Yes — sandbox
Skills / Plugins	No	Via MCP servers	Yes — .zip install
This workflow	Read outputs	Core execution engine	Recommended for use

PART 2

Installation Guide

Windows — step-by-step setup

Prerequisites

Before installing Claude tools on Windows

1 Windows 10 or 11

64-bit required. No WSL needed for Cowork; WSL2 recommended for Claude Code power users.

2 Node.js 18+

Required for Claude Code. Download from nodejs.org — choose LTS version. Verify: `node --version`

3 Anthropic Account

Create a free account at claude.ai. A paid plan (Pro or Team) is required for Cowork access.

4 Python 3.10+

Required for the RAN trial scripts. Download from python.org. Add to PATH during install.

5 Python packages

`pip install xlsxwriter pandas openpyxl python-docx --break-system-packages`. Can be installed by Cowork.

6 Git (optional)

Useful for version-controlling trial reports and skill updates. git-scm.com

Installing Claude Cowork

Windows desktop application

1. **Download the installer**

   Go to claude.ai → Download → Windows. The file is ClaudeSetup.exe (~150 MB).

2. **Run the installer**

   Double-click ClaudeSetup.exe. Accept prompts. Claude installs to %AppData%\Claude. No admin rights needed.

3. **Sign in with Anthropic account**

   Launch Claude from the Start menu. Click "Sign in" and authenticate with your claude.ai credentials.

4. **Enable Cowork mode**

   In the Claude app, look for the Cowork toggle (currently a Research Preview feature). Enable it.

5. **Connect a workspace folder**

   Click "Select Folder" in Cowork mode. Choose the folder where your trial data lives (e.g. Reporting/).

6. **Install the RAN Trial skill**

   Import ran-trial-production_cowork.zip via the Plugins / Skills menu. The skill appears in Claude's available-skills list.

Installing Claude Code

Windows — terminal-based agent

Claude Code runs in the terminal. For Windows, the recommended approach is to install it natively using cmd or PowerShell.

```
# Step 1: Install Node.js 18+ from nodejs.org
# Then verify:
node --version   # should be 18+
npm --version

# Step 2: Install Claude Code
npm install -g @anthropic-ai/claude-code

# Step 3: Launch
cd C:\YourProject
claude
```

Tip: Claude Code works best in the folder that contains your data files. Always cd into the project directory first.

Windows Terminal	Recommended over cmd.exe. Install from Microsoft Store for a better experience.
WSL2 (optional)	Linux subsystem gives access to apt packages. Useful if you need poppler, LibreOffice for PDF exports.
First run	claude will prompt you to authenticate. Follow the browser link to link your Anthropic account.
API key mode	Alternatively: set ANTHROPIC_API_KEY env var in Windows Settings → System → Advanced → Environment Variables.

PART 3

Cowork vs Claude Code

Choosing the right tool for the job

Cowork vs Claude Code — Key Differences

Claude Cowork	Claude Code
Desktop GUI — no terminal needed	Terminal / CLI — developer-centric
Folder selected via file picker	Works in any directory you cd into
Skills installed as .zip bundles	Extensions via MCP servers / plugins
Persistent memory across sessions	Memory only within session (by default)
Sidebar renders Excel, HTML, images	Writes files; you open them yourself
Guided UX — asks clarifying questions	More direct — executes what you ask
Ideal for: analysts, RF engineers	Ideal for: developers, automation scripting
Cowork runs the same scripts via sandbox	Claude Code runs scripts via your shell

VS

Which Tool for the RAN Trials Workflow?

Both tools can run this workflow. The recommended path for RF engineers is **Claude Cowork** — it requires no terminal knowledge and provides a guided experience.

RF Engineer (analyst) **Claude Cowork**	Guided prompts, file picker, no terminal. Install the .zip skill, connect your Reporting folder, and describe what you need.
Developer / Automation **Claude Code**	Full shell control. Pipe data directly, chain scripts, integrate with CI or batch jobs. More flexible, less guardrails.
Either — no preference **Cowork recommended**	Cowork's Skills system means the methodology and reference files are automatically loaded — reducing the chance of analytical errors.

PART 4

Skills & Project Structure

How Claude understands your workflow

What is a Skill?

Installable knowledge + scripts bundles for Claude

A Skill is a folder of files that teaches Claude how to do a specific type of work. When you invoke a skill, Claude reads the SKILL.md file first — which contains instructions, procedures, and references — before taking any action.

`SKILL.md`	Master instruction file. Step-by-step procedure, pitfalls, expected outputs. Claude reads this first every time.
`scripts/`	Python and Node.js scripts Claude runs to generate Excel, charts, and Word documents. One set per RAT (LTE / NR).
`references/`	Methodology files (sigma math, verdict rules, causal-chain principles), KPI column maps, carrier allocations.
`feature_context.json`	Per-trial JSON generated by Claude from the Nokia DN PDF. Contains the KPI causal chain for that specific feature.

```
---
name: ran-trial-production
description: >
  End-to-end production pipeline
  for RAN optimization trial
  analysis. Multi-RAT: 4G LTE and
  5G NR NSA/EN-DC. Use whenever the
  user provides KPI statistics from
  a Nokia RAN trial and wants the
  full deliverable set.
---
# RAN Trial Production Pipeline
1. KPI Charts Excel
   <TRIAL_ID>_<RAT>_KPI_Grouped.xlsx
2. Statistical Analysis Excel
3. Technical Memo (.docx)
```

Project Folder Structure

How files are organised on disk

```
ran-trial-production/        <- skill root
│
├── SKILL.md                  <- master procedure
│
├── scripts/
│   ├── lte/
│   │   ├── build_stats_report_template.py
│   │   ├── build_kpi_charts_template.py
│   │   ├── extract_stats.py
│   │   ├── trial_memo_template.js
│   │   ├── feature_context.json  <- per-trial
│   │   └── feature_context_default.json
│   └── nr/
│       ├── build_stats_report_nr.py
│       ├── build_kpi_charts_nr.py
│       ├── extract_stats.py
│       ├── trial_memo_nr.js
│       ├── feature_context.json  <- per-trial
│       └── feature_context_example.json
│
└── references/
    ├── methodology.md          <- shared
    ├── lte/
    │   ├── methodology_lte.md
    │   └── carrier_allocation.md
    └── nr/
        ├── methodology_nr.md
        ├── kpi_column_map.md
        └── carrier_allocation.md
```

Your Reporting folder (workspace):

```
Reporting/
├── NR_KPI_export_*.xlsx       <- KPI data
├── NR_ES_cluster_report_*.xlsx <- ES (NR)
├── <TRIAL_ID>_NR_KPI_Grouped.xlsx  <- output
├── <TRIAL_ID>_NR_Statistical_Analysis.xlsx
├── <TRIAL_ID>_NR_Trial_Analysis.docx
└── ran-trial-production_cowork.zip
```

The Reporting folder is what you connect to Claude in Cowork mode. Claude reads inputs and writes all outputs here.

feature_context.json is generated by Claude from the Nokia DN PDF. It defines which KPIs to plot in the causal-chain trajectories sheet — no manual script editing needed.

How Claude Operates the Workflow

Logical flow when you trigger the skill

1 **You describe the task**

"Generate the statistical analysis for the <TRIAL_ID> NR trial"

2 **Claude reads SKILL.md**

Full procedure, methodology references, and script documentation are loaded into context

3 **Claude reads methodology files**

methodology.md, methodology_nr.md — sigma rules, verdict criteria, causal-chain guidance

4 **Claude reads your Nokia DN PDF**

Extracts the feature mechanism and KPI causal chain, generates feature_context.json

5 **Claude runs extract_stats.py**

Reads Nokia KPI export, computes baseline/trial means, sigma significance, populates data arrays

6 **Claude runs build_stats_report_nr.py**

Generates 7-sheet Statistical Analysis Excel with 21 charts including causal-chain KPI_Trajectories

7 **Claude runs trial_memo_nr.js**

Generates Word .docx — executive summary, carrier analysis, BTS sanity check, verdict

8 **Claude verifies outputs**

Checks sheet counts, flags warnings, presents files for your review

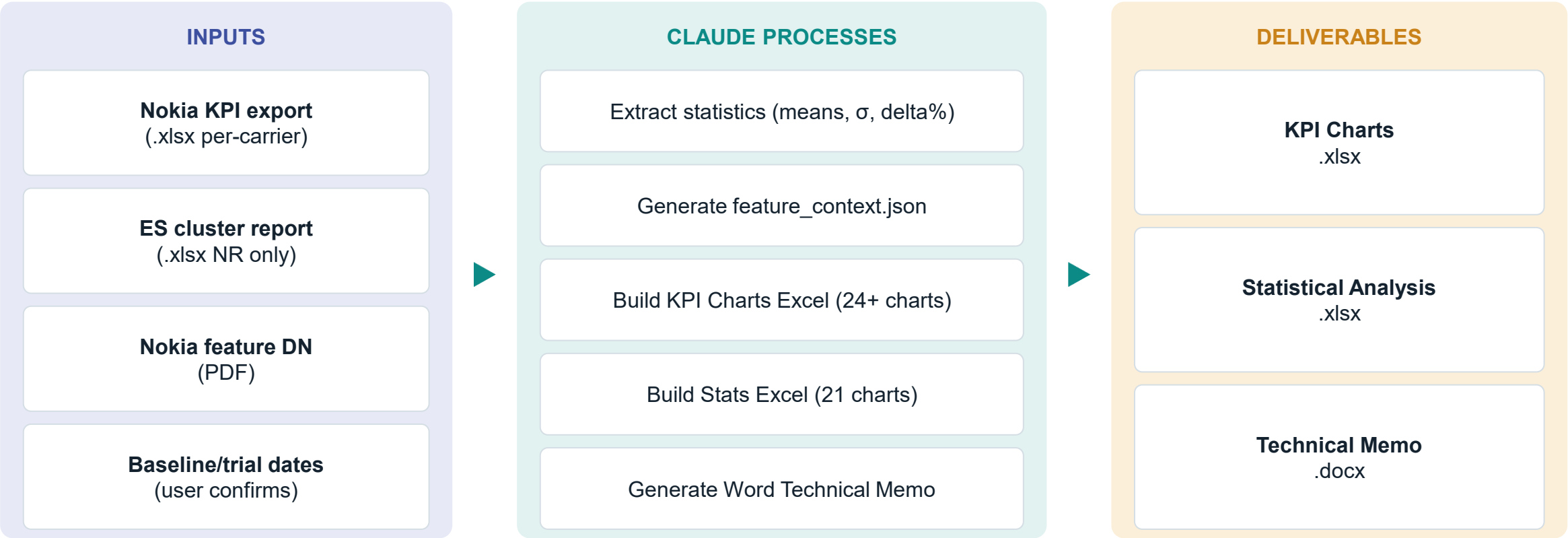
PART 5

RAN Trial Workflow

LTE and 5G NR NSA/EN-DC pipelines

End-to-End Pipeline Overview

Three deliverables from one data export



The full pipeline runs in ~5-7 minutes of compute time. The largest effort is reviewing the outputs and filling in the KPI data arrays from extract_stats.py results.

LTE Workflow — Key Concepts

4G RAN trial specifics

Two-RC structure (regional areas)

RC-A / RC-B

RC-A and RC-B are regional cluster areas — typically RC-A covers Dense Urban and RC-B covers Suburban/Rural. Each area is analysed independently. The SW release may be the same or different.

Band Stratification (optional)

F / U split

Feature Bands (EARFCNs where the parameter changed) vs Unaffected Bands (same cluster, parameter NOT changed). Every KPI table shows both groups separately.

H-Hypothesis Framework

H1/H2/H3

H1: Feature-induced (feature bands degrade, unaffected stable). H2: Concurrent network trend (both degrade). H3: RC-specific SW interaction (one RC unaffected band degrades).

KPI_Trajectories sheet

JSON driven

Bar charts (Baseline / Trial / Post-RB) for the feature causal-chain KPIs. KPI list loaded from feature_context.json — no hardcoded list in the script.

Primary + optional ES file

Feature-specific

Core KPIs (including energy: DRX, PSM, ReducedTX) come from the Nokia KPI export. Some features have supplementary energy exports — include them in the intake form if available.

eNB Sanity Check (optional)

Section 7

Per-eNB consistency check (Extension 2): must show ≥80% of eNBs responding in expected direction on Feature bands. Outlier eNBs flagged with z-score.

H-Hypothesis Framework

Attributing retainability changes to feature, network, or software

When a retainability KPI degrades during a trial, three hypotheses must be evaluated before attributing it to the feature under test.



STOP

Feature did NOT cause it

Feature bands show no retainability / quality change. Do not attribute the degradation to the feature.

H1

Feature-induced (most likely)

Feature bands degrade / Unaffected bands stable. Strongest evidence for feature causation. Required before FAIL verdict is issued.

H2

Concurrent network trend

Both band groups degrade similarly. Likely external cause (traffic shift, outage, weather). Do NOT attribute to feature. Downgrade verdict by one tier.

H3

RC-specific SW interaction

Only one RC's unaffected bands degrade. Issue is in the SW version, not the feature itself. Attribute to SW, base verdict on the clean RC only.

H0

No control group available — check this before Q2. No same-technology unaffected bands in cluster (all FDD affected / TDD not a valid control), so Q2 and Q3 cannot be answered. Fall back to baseline-only comparison. Add explicit verdict caveat.

H-Framework Limitations

What to do when no valid in-cluster control group exists

A control group (unaffected band) must be of the same technology and duplex mode as the feature band. FDD changes cannot use TDD carriers as control — they behave differently (scheduling, interference, load patterns).

LTE — All FDD bands affected

H0 scenario

- Parameter change applied to ALL FDD bands in the cluster
- TDD carriers are present but NOT a valid control
- Band Stratification Not Applicable — document this explicitly
- Baseline vs trial comparison is the only in-cluster evidence
- Check external cluster or prior trial reports for reference
- Verdict: PASS WITH CONDITIONS minimum — cannot exclude H2 from cluster data alone

NR — Feature on all NR carriers

Departure 2

- Cell-level feature activation flag: all NR cells activate simultaneously
- No in-cluster carrier unaffected — H1/H2 indistinguishable from cluster data
- Mitigation: check pre-trial baseline trend stability (flat baseline → H2 less likely)
- External reference cluster: strongest mitigation if available (same market, no feature)
- BTS sanity check (Section 7): ≥80% of BTSs in expected direction supports H1
- Default verdict: PASS WITH CONDITIONS; upgrade to PASS with external evidence

NR / EN-DC Workflow — Key Concepts

5G NR NSA trial specifics — based on an example NR feature trial

No band stratification

Departure 1

The feature activation flag is a cell-level toggle applied to ALL NR cells — no in-cluster unaffected control group. Carrier-only stratification (e.g. NR-LB / NR-MB_F1/F2/F3).

H1/H2 indistinguishable

Departure 2

NR activates the feature on ALL carriers — no in-cluster unaffected control. H1 vs H2 requires external reference cluster. Stable pre-trial baseline supports H1 but does not prove it.

Two data sources

Departure 4

Main KPI export (per-carrier, daily, 100+ columns) + separate ES cluster report (cluster-level only, daily energy counters). Scripts accept both independently.

KPI_Trajectories — daily line charts

Departure 6

NR shows daily line charts per carrier for causal-chain KPIs (LTE uses period bar charts, same sheet name). Feature-context JSON drives both carrier KPIs and ES energy KPIs.

BTS Sanity Check (mandatory)

Departure 7

Per-BTS coverage metric: ≥80% of BTSs must show expected primary KPI direction. z-score >2.5 flags outliers. Populates Section 7.1 of the Word memo.

Verdict caveat without control group

Departure 5

Without in-cluster unaffected carriers, a concurrent network event could explain KPI shifts. Default to PASS WITH CONDITIONS. Upgrade to PASS only when an external reference cluster was reviewed and showed no coincident trend (some work is required).

feature_context.json — The Dynamic KPI Engine

How Claude picks the right KPIs for each trial

Instead of hardcoding a KPI list in the script, Claude generates a feature_context.json file by reading the Nokia feature document (DN PDF) for each trial. The script reads this file at runtime.



Format differences: NR vs LTE

Field	NR feature_context.json	LTE feature_context.json
t1_carrier_kpis	Required — raw data column names	Not used — uses t1_kpis instead
t1_kpis	Not used	Required — names matching RC-A/RC-B_FEAT rows
t1_es_kpis	Required — ES cluster column names	Required — ES cluster column names
higher_bad per KPI	Stored in kpi_column_map.md	Required in JSON — used for bar colours
If JSON absent	KPI_Trajectories sheet skipped	Falls back to static KPIs configuration

Statistical Methodology — Sigma Convention

How significance is measured across all trials

All KPI significance is measured in sigma (σ) — how many standard deviations the trial-period mean is from the baseline mean. This normalises across KPIs with very different absolute scales.

```
σ = (trial_mean − baseline_mean) / baseline_std_dev
baseline_std_dev = std dev of daily values
                   during the baseline period
Chart: positive σ always = degradation
```

Why σ rather than Δ%?

Delta% is scale-dependent — a 0.1 pp change on RRC SR (~100%) is noise; the same 0.1 pp on ERAB Retain. Fail (~0.5%) is critical. Sigma normalises by each KPI's own baseline variability.

≥ 3σ	CRITICAL	Very unlikely to be noise — warrants immediate investigation
≥ 2σ	HIGH	Statistically notable — directionally strong finding
≥ 1σ	MEDIUM	Directional signal — treat as hypothesis, not conclusion
< 1σ	NOISE	Within normal baseline fluctuation — not actionable

±1σ = 68% of days (within normal range) | ±2σ = 95% | ±3σ = 99.7% → only ≥3σ is genuinely unlikely to be baseline noise. Short baseline (7 days): std dev has wide uncertainty — treat 1–2σ as directional only. Assumes daily KPI values are approximately normally distributed during baseline.

PART 6

SKILL & Methodology Files

File-by-file walkthrough

SKILL.md — The Master Procedure File

Claude reads this before every action

SKILL.md is the brain of the workflow. When you invoke the ran-trial-production skill, Claude reads SKILL.md first and follows its procedures for every step.

Step 0	**Select RAT**	LTE or NR — routes to correct scripts and references	
Step 1	**Pre-flight checks**	6 required inputs: trial dates, cluster scope, Nokia exports, feature DN PDF, RC labels, band scope	
Step 1a	**Causal chain extraction**	Read Nokia DN PDF → classify KPIs → generate feature_context.json (NR & LTE)	
Step 2	**Run extract_stats.py**	Compute baseline/trial means, delta%, sigma — populate data arrays for next scripts	
Step 3	**Generate KPI Charts Excel**	24 sheets / 49 charts for NR; similar for LTE — daily trends with period shading	
Step 4	**Generate Statistical Analysis Excel**	7 sheets / 21 charts for NR — sigma matrix, ranking, per-carrier, causal-chain trajectories	
Step 4a	**Generate feature_context.json**	From Nokia DN PDF for both RATs — dynamic KPI selection, no script editing needed	
Step 5	**Generate Technical Memo (.docx)**	Word report: exec summary, band scope, KPI sections 4.1–4.5, verdict, BTS sanity check	
Step 5a	**BTS / eNB Sanity Check**	Per-BTS coverage ≥80%, z-score outlier detection — mandatory before finalising memo	
Step 6	**Verification**	Sheet count check, warn on missing KPIs, present deliverables for review	

Methodology Reference Files

The analytical rulebook Claude follows

methodology.md

Shared — both RATs

- Sigma formula and significance thresholds (1/2/3σ)
- Near-zero variance handling (σ not meaningful → report absolute delta)
- Chart sigma normalisation: positive = degradation always
- H-hypothesis framework (H1/H2/H3) for retainability
- Verdict mapping: PASS / PASS WITH CONDITIONS / INCONCLUSIVE / FAIL
- QCI stratification and two-RC trial structure

methodology_lte.md

LTE specific extensions

- Band stratification: Feature vs Unaffected EARFCNs
- H-framework applied to LTE retainability analysis
- feature_context.json procedure for LTE (t1_kpis, higher_bad required)
- eNB sanity check: Feature band cells vs Unaffected band cells per eNB
- Verdict calibration: H2 downgrades one tier; H3 uses unaffected RC only
- Default EARFCN mappings for the reference cluster (confirm before use)

methodology_nr.md

NR / EN-DC departures

- Departure 1: No Feature/Unaffected band split (cell-level parameter)
- Departure 2: H-framework not applicable — state confounder caveat in memo
- Departure 4: Two data sources (main KPI export + ES cluster report)
- Departure 6: feature_context.json — causal chain from Nokia DN PDF
- Departure 7: BTS sanity check mandatory — ≥80% coverage threshold
- Feature-specific notes (e.g. ReducedTX = 0% → PA full shutdown not active in that SW release)

PART 7

How to Use the Workflow

Running the pipeline and what to expect

Running Your First Trial Analysis

Step-by-step user guide

1 **Prepare your data folder** — 5 min

Copy Nokia KPI exports (.xlsx) and ES cluster report (.xlsx, NR only) into your Reporting folder. Also have the Nokia DN PDF on hand.

2 **Open Claude Cowork, connect folder** — 2 min

Launch Claude, enable Cowork mode, select your Reporting folder. Verify the ran-trial-production skill shows in available skills.

3 **Attach the Nokia DN PDF and describe the trial** — 2 min

Say: "Generate the statistical analysis for [TRIAL_ID] NR trial. The baseline is [dates], trial is [dates]." Attach the DN PDF.

4 **Claude extracts causal chain & generates feature_context.json** — Auto ~1 min

Claude reads the DN PDF, identifies the mechanism, classifies KPIs into 4 categories, and writes feature_context.json to the script directory.

5 **Claude runs extract_stats.py** — Auto ~1 min

Reads your Nokia export, computes all statistics. You may be asked to confirm which rows represent baseline vs trial periods.

6 **Review outputs** — 30-60 min

Three files appear in your Reporting folder: KPI Charts Excel, Statistical Analysis Excel, Technical Memo (.docx). Review and fill in any narrative gaps.

What the Outputs Look Like

Three deliverables in detail

`KPI_Grouped.xlsx`

Trend charts for all KPIs grouped by category

- 24+ sheets for NR (D_ data + C_ chart pairs)
- One line per carrier (e.g. 4 NR carriers)
- Daily time series across full trial window
- Trial period highlighted in yellow shading
- Red marker line at trial start date
- LTE: Feature Band + Unaffected Band separate lines

`Statistical_Analysis.xlsx`

Sigma significance matrix and causal-chain trajectories

- Overview: trial metadata summary
- Significance_Matrix: all KPIs with σ, Δ%, tier colour coding
- Sigma_Chart: horizontal bar chart — degradation highlights in red
- Significance_Ranking: sorted by |σ|, watchdogs flagged
- Per_Carrier_Detail: per-carrier sigma breakdown
- Energy_Saving: cluster-level energy counters (NR)
- KPI_Trajectories: causal-chain daily charts from feature_context.json

`Trial_Analysis.docx`

Formal technical memo for stakeholder reporting

- Section 1: Executive Summary with verdict badge
- Section 2: Trial scope, cluster, band/carrier allocation
- Section 3: Data quality assessment
- Section 4: KPI analysis — accessibility, retainability, mobility, energy
- Section 5: Confounding factor analysis
- Section 6: Verdict and conditions
- Section 7: BTS/eNB sanity check findings (quantitative)

Common Pitfalls & How to Avoid Them

✕ **Wrong feature mechanism for KPI_Trajectories**	✓ **FIX:** Always derive feature_context.json from the actual Nokia DN PDF for that trial. μDTX ≠ MIMO sleep ≠ beamforming — completely different KPI signatures.
✕ **Missing feature_context.json before running the script**	✓ **FIX:** NR: KPI_Trajectories sheet is skipped silently. LTE: script falls back to the bundled default feature profile. Always check the log message at startup.
✕ **File open in Excel when script tries to write**	✓ **FIX:** Close the output .xlsx file before re-running. The script fails at wb.close() if the file is locked by Excel.
✕ **Near-zero baseline variance exploding sigma**	✓ **FIX:** When a KPI was constant in baseline (e.g. RRC SR = 100%), sigma is not meaningful. Report absolute delta instead. Flag in memo legend.
✕ **Attributing unaffected-band degradation to the feature (LTE)**	✓ **FIX:** If a KPI degrades on UNAFFECTED bands, it cannot be the feature's fault. Run the H-framework check before asserting H1.
✕ **Claiming PASS when confounders not excluded (NR)**	✓ **FIX:** Without an unaffected control group, the default NR verdict is PASS WITH CONDITIONS. Only escalate to PASS if user confirms external control reviewed.

PART 8

Tips & Next Steps

Best practices and extending the workflow

Best Practices

1

One folder per trial

Keep each trial's Nokia exports, feature_context.json, and deliverables in its own dated folder. Prevents confusion when re-running.

2

Always provide the DN PDF

Without it, Claude cannot derive the causal chain. The KPI_Trajectories sheet will be skipped or will show the wrong KPIs for the wrong feature class.

3

Confirm dates explicitly

Do not rely on Claude to infer the baseline/trial boundary from filenames. State the exact dates, e.g. "Baseline: 2026-01-05 to 2026-01-18."

4

Run extract_stats.py first, review before proceeding

Check the statistics output before generating charts and memo. A sigma that looks implausible often signals a date range or column name issue.

5

Check BTS sanity before finalising verdict

Cluster-level KPI improvements can hide site-level heterogeneity. If <80% of BTSs respond as expected, the verdict must include a condition.

6

Update the skill after methodology changes

If you change the analytical approach (new KPI, different sigma threshold), update methodology_lte.md / methodology_nr.md and reinstall the .zip skill.

Extending to a New Trial

Checklist for onboarding a new trial to the workflow

The workflow is designed to require minimal changes between trials. Most customisation happens at the data layer (new Nokia exports) and the knowledge layer (feature_context.json), not the code.

Always required	Generated by Claude (automatic)	Manual if different feature class
New Nokia KPI export .xlsx for the trial cluster	feature_context.json from the DN PDF	Update feature_context.json if µDTX → MIMO sleep
New Nokia DN PDF (feature document)	Statistics arrays (extract_stats.py output)	Verify KPI column names in kpi_column_map.md match new export
Confirmed baseline and trial start/end dates	KPI_Grouped.xlsx (charts auto-built)	Update carrier_allocation.md if different cluster bands
RC label(s) — e.g. RC-A, RC-A/RC-B	Statistical_Analysis.xlsx (all 7 sheets)	Add new LTE EARFCN → band mappings if needed
Cluster name and band/carrier scope	Trial_Analysis.docx (Word memo)	Update verdict guidance in methodology_nr.md if acceptance criteria differ

Summary

- Claude Cowork + Skills = guided, repeatable AI workflow with no terminal
- feature_context.json makes KPI selection dynamic — no script editing per trial
- LTE and NR pipelines share the same JSON pattern with RAT-specific adaptations
- Sigma methodology + H-framework + BTS sanity check = statistically sound output
- Three deliverables: KPI Charts Excel, Statistical Analysis Excel, Word Memo

Questions?

ran-trial-production_cowork.zip
available on request

Andriy Korobeyko · 2026